

OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop harnesses Machine Learning and AI to provide intelligent crop recommendation and agricultural production optimization, offering farmers and agricultural professionals data-driven insights for maximizing crop yield. The platform includes a Crop Prediction module, which analyzes soil and environmental parameters to recommend the most suitable crops, a Parameter Validation system that ensures input accuracy, and a Results Dashboard that displays predicted crops with confidence scores and farming recommendations.

Utilizing a trained Random Forest/Decision Tree model with scikit-learn, OptiCrop processes user inputs including Nitrogen (N), Phosphorus (P), Potassium (K), temperature, humidity, pH, and rainfall to deliver accurate crop recommendations. Built with Flask and deployed publicly via Render, the platform ensures a seamless, responsive, and user-friendly experience. With secure model management and robust input validation, OptiCrop empowers farmers, agronomists, and agricultural businesses to make informed planting decisions with confidence.

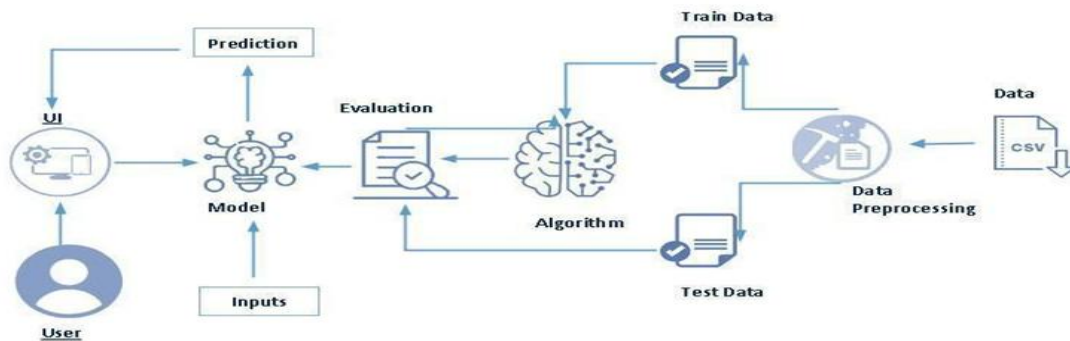
Scenarios

Scenario 1: A farmer in a tropical region enters soil parameters (N=90, P=40, K=45, pH=6.5) and environmental conditions (temperature=27°C, humidity=65%, rainfall=120mm). The system instantly recommends "Rice" as the optimal crop with a 92% confidence score, along with expected yield information and specific farming tips for rice cultivation.

Scenario 2: An agronomist working with a client who has sandy soil with low nitrogen content (N=30, P=25, K=35, pH=5.8) and moderate rainfall uses OptiCrop. The AI recommends "Groundnuts" as the most suitable crop, providing water requirement details, growing season information, and soil amendment suggestions to optimize yield.

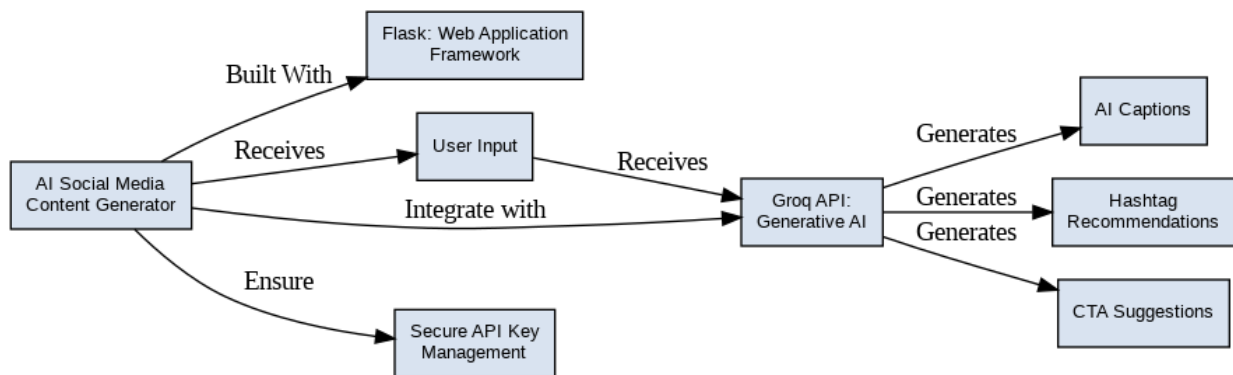
Scenario 3: A large-scale farming cooperative wants to diversify crops across different plots. They input varying parameter sets for five different fields. OptiCrop generates recommendations for Rice, Cotton, Maize, Wheat, and Coffee respectively, allowing the cooperative to plan crop rotation and resource allocation efficiently.

Technical Architecture:



Description: OptiCrop utilizes a modular three-tier architecture to deliver rapid, AI-driven crop recommendations. The frontend provides a responsive user interface built with HTML5, CSS3, and JavaScript featuring modern glassmorphism design elements. The Flask-based backend orchestrates data validation, parameter processing, and model inference. It interfaces with the pre-trained machine learning model (loaded via pickle) to generate tailored crop recommendations, with secure deployment managed locally via Flask and exposed publicly through Render cloud hosting.

System Architecture Diagram:



Pre-requisites:

Flask Framework Knowledge: [Flask Documentation](#)

Machine Learning Fundamentals: [Scikit-learn Documentation](#)

HTML, CSS, and JavaScript Skills: [W3Schools HTML/CSS/JavaScript Tutorials](#)

Python Programming Proficiency: [Python Documentation](#)

Version Control with Git: [Git Documentation](#)

Development Environment Setup: [Flask Installation Guide](#)

Cloud Deployment with Render: [Render Documentation](#)

Project Workflow:

Milestone 1: Model Selection and Architecture

Activity 1.1: Research and select the appropriate machine learning algorithm for crop classification

Activity 1.2: Train and validate the crop recommendation model using agricultural datasets

Activity 1.3: Define the architecture of the application, detailing interactions between the frontend, backend, and ML model

Activity 1.4: Set up the development environment, installing necessary libraries and dependencies for Flask and scikit-learn

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the core functionalities: Crop Prediction Engine, Parameter Validation, and Results Generation

Activity 2.2: Implement the Flask backend to manage routing, user input processing, and model inference

Milestone 3: App.py Development

Activity 3.1: Write the main application logic in app.py, establishing routes for each feature and integrating ML model predictions

Milestone 4: Frontend Development

Activity 4.1: Design and develop the user interface using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout with glassmorphism design

Activity 4.2: Create dynamic templates with Flask's render_template to display prediction results based on user interactions

Milestone 5: Deployment

Activity 5.1: Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed

Activity 5.2: Deploy the application publicly using Render to expose the Flask server over a secure public URL

Milestone 6: Testing and Optimization

Activity 6.1: Conduct comprehensive testing of all features and optimize model performance

Milestone 7: Conclusion

The development of OptiCrop highlights the transformative potential of AI in agriculture. By continuously refining the underlying machine learning models with larger, more diverse datasets and enhancing the frontend accessibility, OptiCrop is well-positioned to evolve from a specialized crop recommendation tool into a comprehensive, end-to-end smart farming management platform.

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate machine learning algorithm for crop classification and recommendation. This involves researching the capabilities and performance of various models, ensuring that the chosen model aligns well with the application's objectives of providing accurate crop recommendations based on soil and environmental parameters.

Milestone 1: Dataset Collection and Model Development

The Crop Recommendation Dataset is collected and preprocessed using Python. Features including Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall are selected as input variables. The Random Forest algorithm is trained using the dataset and the final model is saved as `model.pkl`.

Milestone 2: Backend Development

The backend is developed using Flask. The application handles user authentication, data validation, Machine Learning prediction, and page routing. User registration and login credentials are verified before allowing access to the prediction dashboard.

Milestone 3: Frontend Development

The frontend provides a responsive and easy-to-use interface. Users can register, log in, navigate through different pages, enter agricultural parameters, and receive crop recommendations. The application also includes an Admin Login page for administrator access.

Milestone 4: Machine Learning Integration

The trained Machine Learning model is loaded during application startup. Whenever the user enters valid soil and environmental parameters, the backend processes the input and predicts the most suitable crop. The prediction result is displayed instantly.

Milestone 5: Deployment

The application is deployed on Render using GitHub integration. After deployment, users can access the application through a public URL without installing any software.

Activity 1.2: Define the Application Architecture

Architectural Components:

Start

|



Home Page

|

└───────────────────▶ About Page

|

└───────────────────▶ User Registration

|

|

|



User Account Created

|

└───────────────────▶ User Login

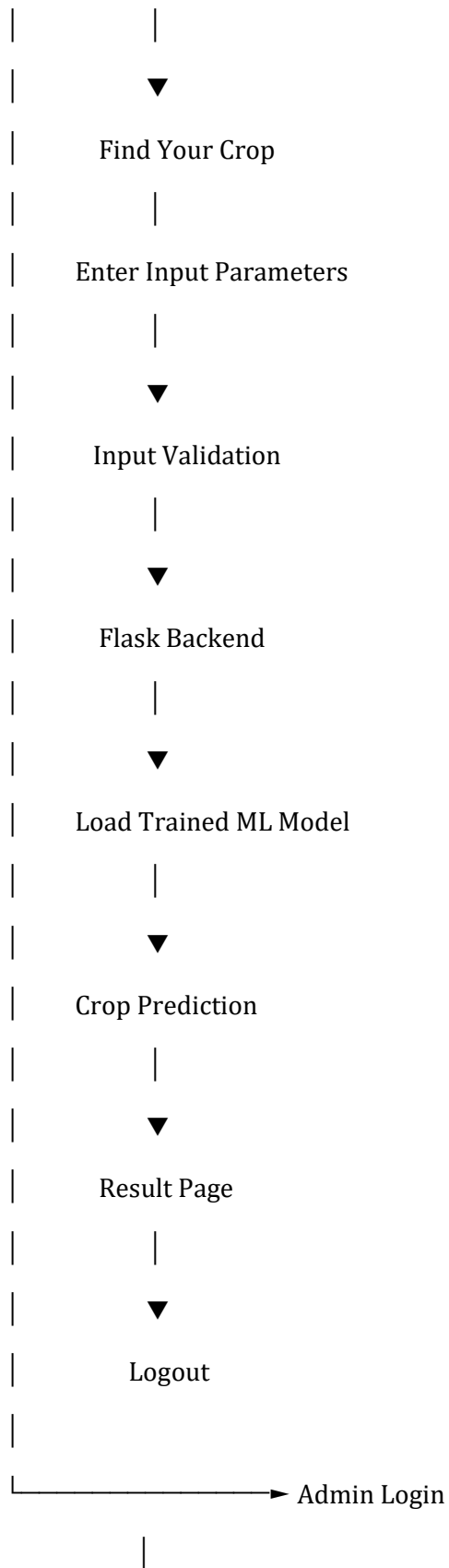
|

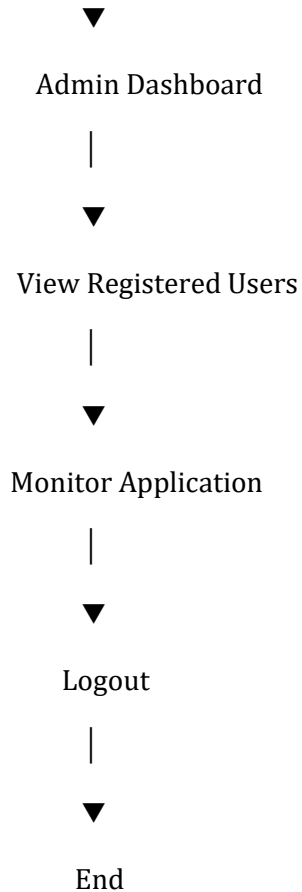
|

|



User Dashboard





Frontend Layer:

HTML5 structure with semantic elements

CSS3 with glassmorphism design and animations

Vanilla JavaScript for dynamic interactions

Responsive design for mobile, tablet, and desktop

Backend Layer (Flask):

Route management (/, /findyourcrop, /predict, /about)

Input validation and sanitization

Model loading and inference

JSON API responses

Machine Learning Layer:

Pre-trained Random Forest model

Feature extraction and preprocessing

Prediction generation

Confidence score calculation

Activity 1.3: Define the Architecture of the Application

Draft an Architectural Diagram

Create a three-tier architecture diagram representing the major components of the OptiCrop application. The architecture consists of the Frontend Layer, Backend Layer, and Machine Learning Layer. The frontend is developed using HTML5, CSS3, and JavaScript to provide an interactive user interface. The Flask backend manages routing, user authentication, input validation, prediction requests, and communication with the trained Machine Learning model. The Machine Learning layer contains the trained Random Forest model (model.pkl), which processes agricultural parameters and predicts the most suitable crop. The application is deployed on Render, enabling users to access the system through a web browser.

Detail Frontend Functionality

The frontend provides a responsive and user-friendly interface for interacting with the application. Users can navigate through the Home, About, User Registration, User Login, Dashboard, Find Your Crop, Result, Admin Login, and Admin Dashboard pages. After logging in successfully, users enter agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH, and Rainfall through the crop prediction form. JavaScript performs basic client-side validation before submitting the data to the Flask backend. The prediction results are then displayed on the Result page in a clear and user-friendly format.

Outline Backend Responsibilities

The Flask backend is responsible for handling all application logic and routing. It manages user registration and authentication, validates user inputs, processes prediction requests, and communicates with the trained Machine Learning model. When the user submits the crop prediction form, the backend receives the request, verifies that all required input values are valid, loads the trained model (model.pkl), performs the prediction, and returns the recommended crop to the frontend. The backend also manages administrator login and dashboard access while ensuring secure communication between the frontend and the prediction engine.

Describe Machine Learning Integration Points

The Machine Learning component is integrated into the Flask backend using the trained Random Forest model stored as model.pkl. When a prediction request is received, the backend extracts the input parameters (Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall), converts them into the required numerical format, and passes them to the trained model. The model analyzes the input values and predicts the most

suitable crop based on patterns learned from the Crop Recommendation dataset (Crop_recommendation.csv). The prediction result is then sent back to the Flask application and displayed on the Result page. This integration enables fast, accurate, and real-time crop recommendations for users

Activity 1.4: Set Up the Development Environment

Installation Steps:

Install Python 3.8+:

```
PS C:\Users\gutti\Downloads\OptiCrop\OptiCrop> python --version
Python 3.14.0
```

Install Dependencies:

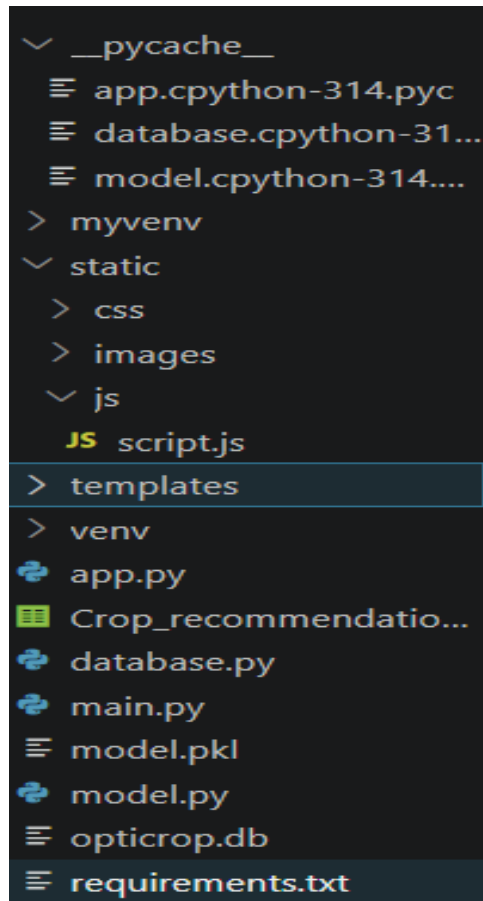
```
(venv) PS C:\Users\gutti\Downloads\OptiCrop\OptiCrop> pip install Flask==3.0.3
>> pip install scikit-learn==1.6.1
>> pip install numpy==2.2.6
>> pip install pandas==2.2.3
>> pip install gunicorn==23.0.0
>> pip install flask-cors==4.0.0
Collecting Flask==3.0.3
  Using cached flask-3.0.3-py3-none-any.whl.metadata (3.2 kB)
Collecting Werkzeug>=3.0.0 (from Flask==3.0.3)
  Using cached werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting Jinja2>=3.1.2 (from Flask==3.0.3)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
```

Create requirements.txt:

```
requirements.txt
1 Flask==3.0.3
2 gunicorn==23.0.0
3 numpy==2.2.6
4 pandas==2.2.3
5 scikit-learn==1.6.1
6 joblib==1.5.1
```

Set Up Application Structure:

Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Prediction Features

Implement the core functionalities of the OptiCrop application, including **User Registration, User Login, Admin Login, Crop Recommendation, and Result Generation**. These modules work together to provide a secure and intelligent crop recommendation system for farmers and agricultural professionals.

User Registration

The User Registration module allows new users to create an account by entering their personal information such as name, email, username, and password. The system validates all mandatory fields before storing the user details securely, ensuring only registered users can access the application.

User Login

The User Login module authenticates registered users using their username or email and password. After successful authentication, users are redirected to the dashboard, where they can access the crop prediction feature.

Crop Recommendation Module

The Crop Recommendation module is the primary feature of the application. Users enter agricultural parameters including Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH value, and Rainfall. The Machine Learning model processes these parameters and predicts the most suitable crop for cultivation.

Result Generation

After processing the user inputs, the system displays the predicted crop on the Result page. The recommendation is generated by the trained Random Forest model and presented in a clear and user-friendly format to assist users in making informed agricultural decisions.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask

Create routes in **app.py** to manage all application pages and functionalities.

Example routes include:

- Home Page (/)
- About Page (/about)
- User Registration (/signup)
- User Login (/signin)
- Dashboard (/dashboard)
- Find Your Crop (/findyourcrop)
- Prediction (/predict)
- Result Page (/result)
- Admin Login (/admin_login)
- Admin Dashboard (/admin_dashboard)

Process User Input

The Flask backend receives agricultural parameters submitted through the crop prediction form. Before prediction, the application validates all input values to ensure they are complete, numeric, and within acceptable ranges.

The following parameters are collected from the user:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH Value
- Rainfall

If invalid or incomplete information is entered, the application displays appropriate validation messages to prevent incorrect predictions.

Integrate Machine Learning Model

After successful validation, the Flask backend loads the trained Machine Learning model (model.pkl) and passes the user inputs for prediction. The model analyzes the agricultural parameters using the Random Forest algorithm and predicts the most suitable crop based on patterns learned from the Crop Recommendation dataset.

The predicted crop is then returned to the Flask application and displayed on the Result page.

Display Prediction Results

The prediction result is presented through an interactive Result page. Users can view the recommended crop immediately after prediction. The backend ensures smooth communication between the frontend and the Machine Learning model, providing fast and accurate recommendations.

Backend Responsibilities

The Flask backend performs the following responsibilities:

- Handle User Registration and Login.
- Authenticate Admin Login.

- Validate all user inputs.
- Manage application routing.
- Load the trained Machine Learning model (model.pkl).
- Perform crop prediction using the Random Forest algorithm.
- Return prediction results to the frontend.
- Display error messages for invalid inputs.
- Render the appropriate HTML pages using Flask templates.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the **app.py** file, which serves as the backbone of the OptiCrop application. In this milestone, the Flask framework is used to define application routes, manage user authentication, validate user inputs, integrate the trained Machine Learning model, and display prediction results. The **app.py** file acts as the communication bridge between the frontend interface and the Machine Learning prediction engine.

Activity 3.1: Writing the Main Application Logic in **app.py**

*Define the Core Routes in **app.py**:*

- Establish routes for OptiCrop two endpoints, each serving a distinct purpose:
 - **/** — Home page route that renders the main index.html interface

```
@app.route("/")
def home():
    trigger_signin = request.args.get("trigger_signin")
    return render_template("home.html", trigger_signin=trigger_signin)
```

- **/ predict** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```

@app.route("/predict", methods=["POST"])
@login_required
def predict():
    if crop_model is None:
        return jsonify({"success": False, "error": "ML Prediction Model is not loaded on the server"}), 500

    try:
        # Support both JSON payload (AJAX) and form submissions
        if request.is_json:
            data = request.get_json()
        else:
            data = {
                "nitrogen": request.form.get("nitrogen"),
                "phosphorus": request.form.get("phosphorus"),
                "potassium": request.form.get("potassium"),
                "temperature": request.form.get("temperature"),
                "humidity": request.form.get("humidity"),
                "ph": request.form.get("ph"),
                "rainfall": request.form.get("rainfall"),
            }

        # Check required fields
        required_fields = ["temperature", "humidity", "rainfall", "ph", "nitrogen", "phosphorus", "potassium"]
        missing = [field for field in required_fields if not data.get(field)]
        if missing:
            return jsonify({"success": False, "error": f"Missing agricultural parameters: {', '.join(missing)}"}), 400

        # Parse and validate ranges
        try:
            temperature = float(data["temperature"])
            humidity = float(data["humidity"])
            rainfall = float(data["rainfall"])
            ph = float(data["ph"])
            nitrogen = float(data["nitrogen"])
            phosphorus = float(data["phosphorus"])
            potassium = float(data["potassium"])
        except ValueError:
            return jsonify({"success": False, "error": "Invalid parameter input. Please enter numeric values."}), 400

        # Make prediction
        if hasattr(crop_model, 'predict') and callable(getattr(crop_model, 'predict')):
            try:
                # Rule-based custom model prediction
                recommendation = crop_model.predict(
                    temperature, humidity, rainfall, ph, nitrogen, phosphorus, potassium
                )
                if isinstance(recommendation, dict):
                    crop_name = recommendation.get("name", "Mixed Vegetables")
                    confidence = recommendation.get("confidence", 85)
                else:
                    crop_name = str(recommendation)
                    confidence = 88
            except TypeError:
                # Sklearn ML model prediction
                features = [nitrogen, phosphorus, potassium, temperature, humidity, ph, rainfall]
                prediction = crop_model.predict([features])
                crop_name = str(prediction[0]).capitalize()
                confidence = 94
        else:
            return jsonify({"success": False, "error": "Invalid recommendation model structure"}), 500

        # Log prediction to SQL database
        prediction_id = db.add_prediction(
            user_id=session["user_id"],
            nitrogen=nitrogen,
            phosphorus=phosphorus,
            potassium=potassium,
            temp=temperature,
            humidity=humidity,
            ph=ph,

```

```
        rainfall=rainfall,
        predicted_crop=crop_name,
        confidence=confidence
    )

    return jsonify({
        "success": True,
        "prediction_id": prediction_id,
        "crop": crop_name,
        "confidence": confidence
    })

except Exception as e:
    print(f"[ERROR] Prediction error: {str(e)}")
    traceback.print_exc()
    return jsonify({
        "success": False,
        "error": str(e)
    }), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Machine Learning Prediction Helpers

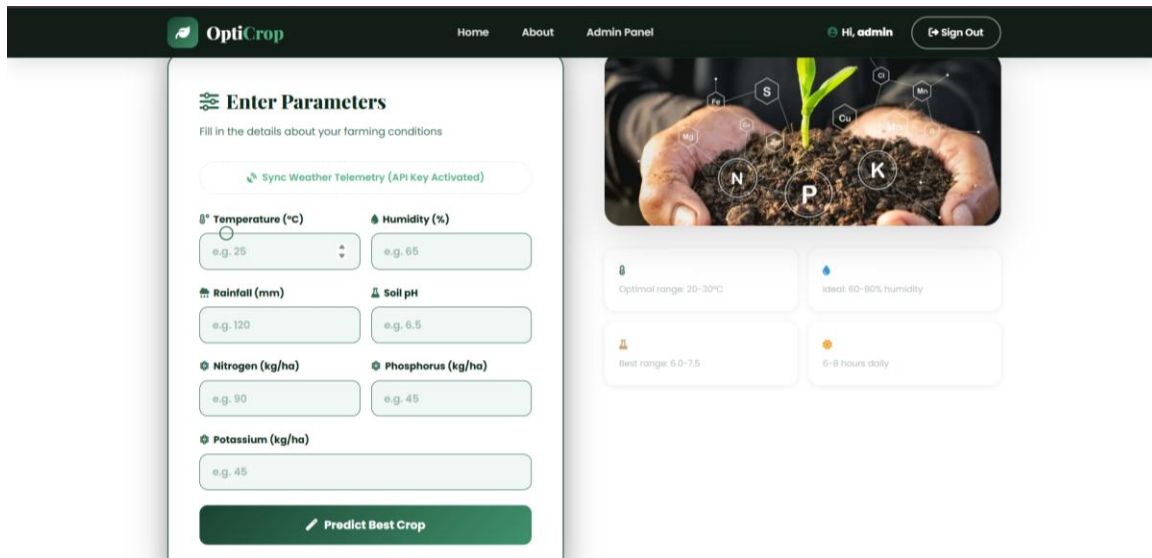
The OptiCrop application uses a trained **Random Forest Machine Learning model** to predict the most suitable crop based on soil nutrients and environmental conditions. The prediction logic is implemented in the Flask backend and uses the trained model stored as **model.pkl**.

Input Parameters

The application accepts the following agricultural parameters from the user:

- Nitrogen (N)
- Phosphorus (P)
- Potassium (K)
- Temperature
- Humidity
- pH Value
- Rainfall

These values are collected through the **Find Your Crop** page and validated before prediction.

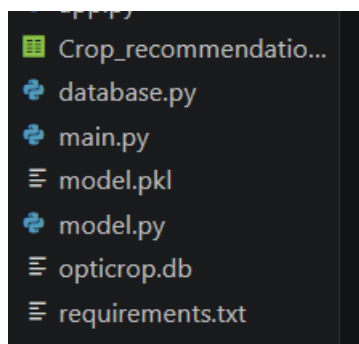


The screenshot shows the OptiCrop web application interface. The header includes the OptiCrop logo, navigation links (Home, About, Admin Panel), and user information (Hi, admin, Sign Out). The main content area is divided into two sections. The left section, titled "Enter Parameters", prompts users to "Fill in the details about your farming conditions" and includes a "Sync Weather Telemetry (API Key Activated)" button. Below this, there are input fields for Temperature (°C), Humidity (%), Rainfall (mm), Soil pH, Nitrogen (kg/ha), Phosphorus (kg/ha), and Potassium (kg/ha). Each field has a placeholder value (e.g., 25, 65, 120, 6.5, 90, 45). A green "Predict Best Crop" button is at the bottom of this section. The right section features a large image of hands holding soil with nutrient labels (N, P, K, S, Ca, Mg, Cu, Zn, Fe) and a small plant. Below the image are four cards displaying optimal ranges: "Optimal range: 20-30°C", "Ideal: 60-80% humidity", "Best range: 6.0-7.5", and "6-8 hours daily".

Model Loading

The Flask application loads the trained Machine Learning model during application startup.

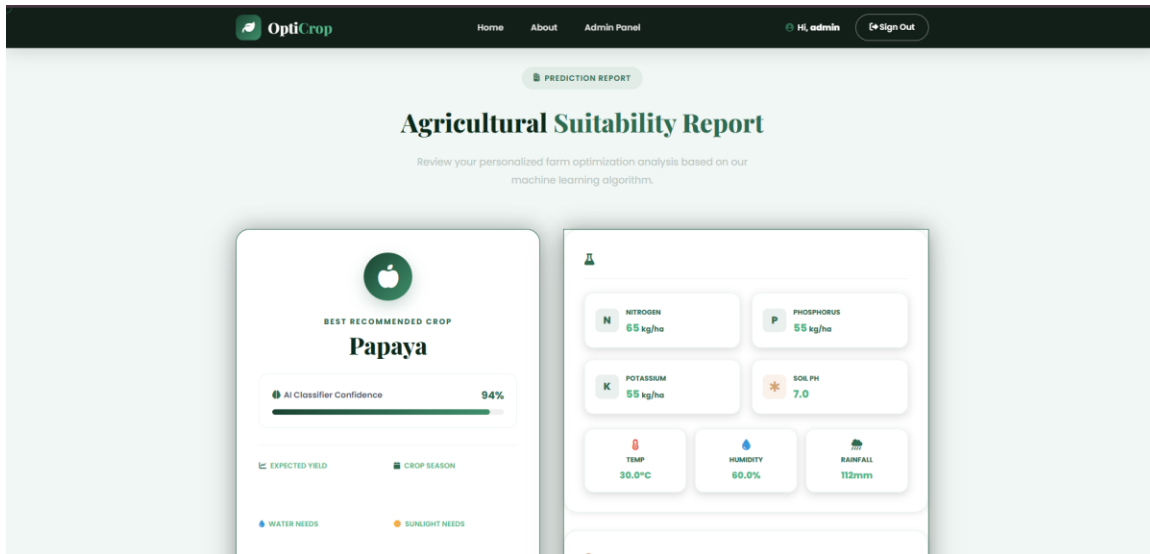
The model file (**model.pkl**) is loaded only once and reused for all prediction requests, improving the overall performance of the application.



Prediction Function

After validating the user inputs, the Flask backend converts the values into numerical format and passes them to the trained Random Forest model.

The prediction function analyzes the given soil and climate conditions and returns the most suitable crop.



Integrate the Machine Learning Model in the Application

The trained Machine Learning model is integrated with the Flask backend to provide real-time crop recommendations.

Whenever a user submits the prediction form, the Flask application performs the following steps:

- Receives the agricultural parameters.
- Validates all input values.
- Loads the trained Random Forest model.
- Predicts the recommended crop.
- Sends the prediction result to the Result page.

This integration enables accurate and efficient crop prediction without requiring model retraining.

Validate Prediction Requests

Before performing prediction, the backend verifies that:

- All required fields are entered.
- Input values are numeric.
- Empty values are not accepted.
- Invalid data is rejected with an appropriate error message.

This validation improves the reliability and accuracy of the prediction process.

Return Prediction Results

After successful prediction, the backend sends the recommended crop to the frontend.

The Result page displays the predicted crop in a user-friendly format, helping farmers make informed agricultural decisions.

Milestone 4: Frontend Development

Milestone 4 focuses on designing and developing an interactive, responsive, and user-friendly interface for the OptiCrop application. The frontend is developed using **HTML, CSS, and JavaScript** to provide seamless navigation and an improved user experience. The application includes separate pages for Home, About, User Registration, User Login, Dashboard, Crop Prediction, Result, Admin Login, and Admin Dashboard.

Activity 4.1: Designing and Developing the User Interface

Home Page

The Home page introduces the OptiCrop application and provides navigation to all major modules.

About Page

The About page explains the project objectives, technologies used, and the benefits of AI-based crop recommendation.

User Registration Page

Allows new users to register securely by entering personal details.

User Login Page

Authenticates registered users and redirects them to the Dashboard.

Dashboard

The Dashboard provides quick navigation to the crop prediction module and other application features.

Find Your Crop Page

Users enter:

- Nitrogen
- Phosphorus
- Potassium
- Temperature
- Humidity
- pH
- Rainfall

After entering the values, users click the **Predict** button.

Result Page

Displays the crop predicted by the Machine Learning model.

Admin Login Page

Allows administrators to log in securely.

Admin Dashboard

Displays administrative information and allows administrators to monitor the application.

Activity 4.2: Dynamic Content Rendering

The application dynamically displays prediction results after receiving the response from the Flask backend.

JavaScript is used to validate the form before submission and improve user interaction.

After prediction, the Result page displays the recommended crop without requiring additional processing from the user.

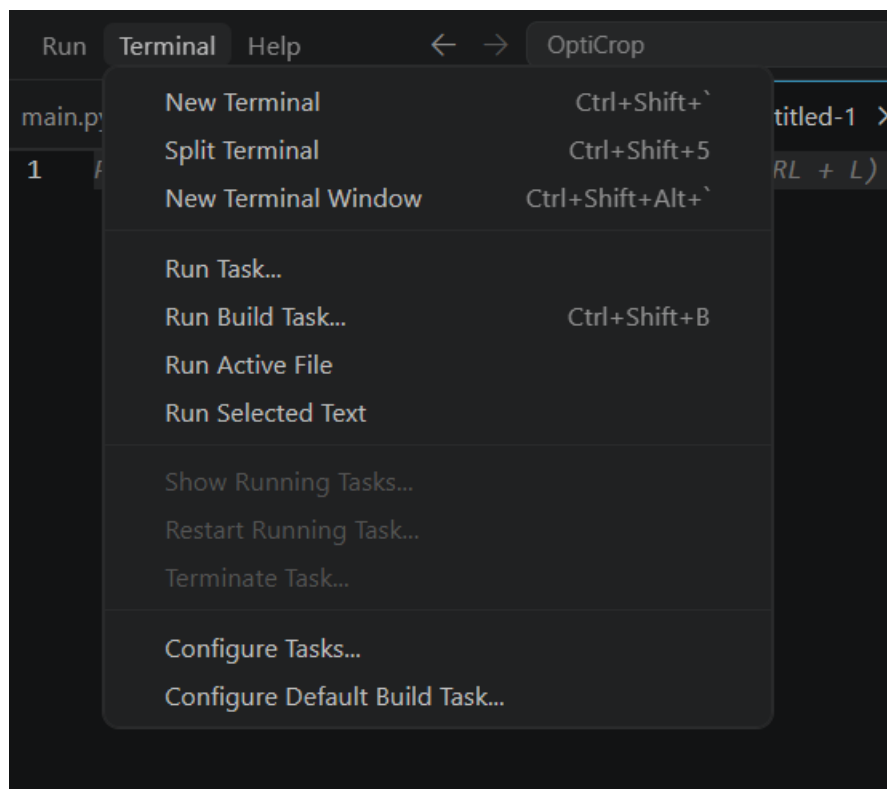
Milestone 5: Deployment

Milestone 5 focuses on deploying the OptiCrop application so that it can be accessed publicly through a web browser. The project is deployed using **Render**, allowing users to access the application online without installing any software.

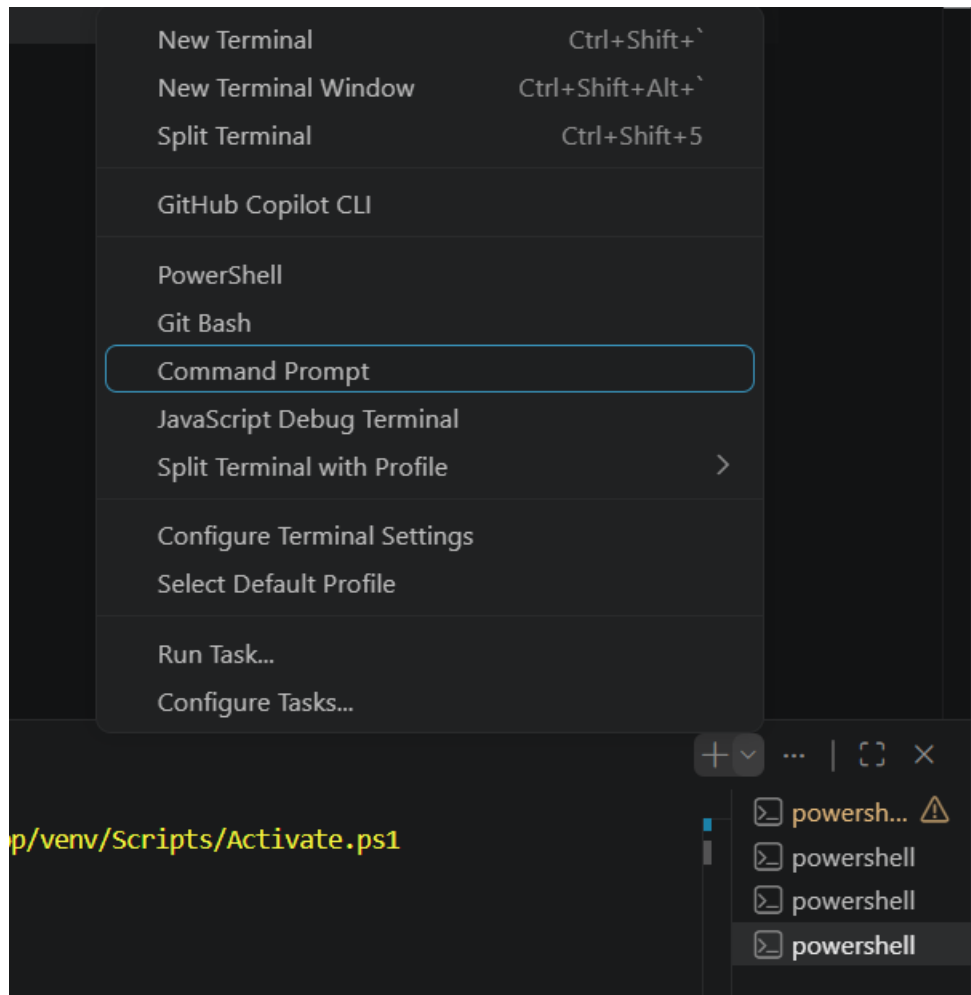
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

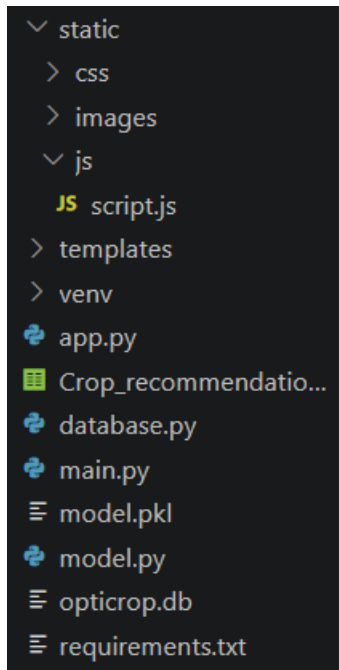
- Open a New Terminal



- Then open “Command Prompt”



- Verify that the following files are available:
 - app.py
 - model.pkl
 - Crop_recommendation.csv
 - requirements.txt
 - templates/
 - static/
- Test the application locally before deployment.



Activity 5.2: Deploy the Application on Render

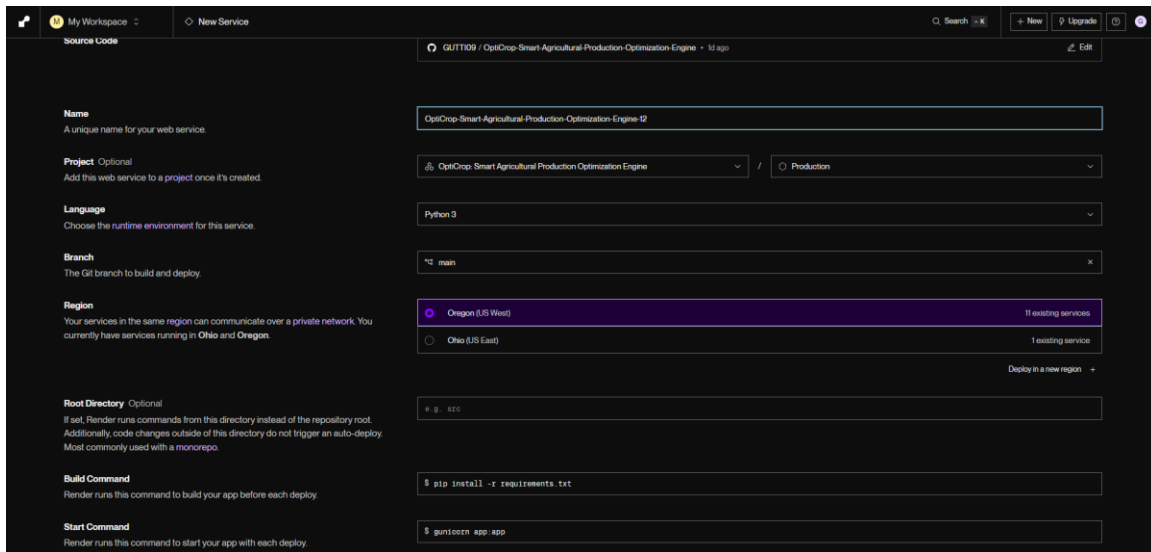
- Push the project to GitHub.
- Connect the GitHub repository with Render.
- Configure the build command:

```
pip install -r requirements.txt
```

- Configure the start command:

```
gunicorn app:app
```

- Deploy the application.
- Verify that the application is accessible using the Render URL.



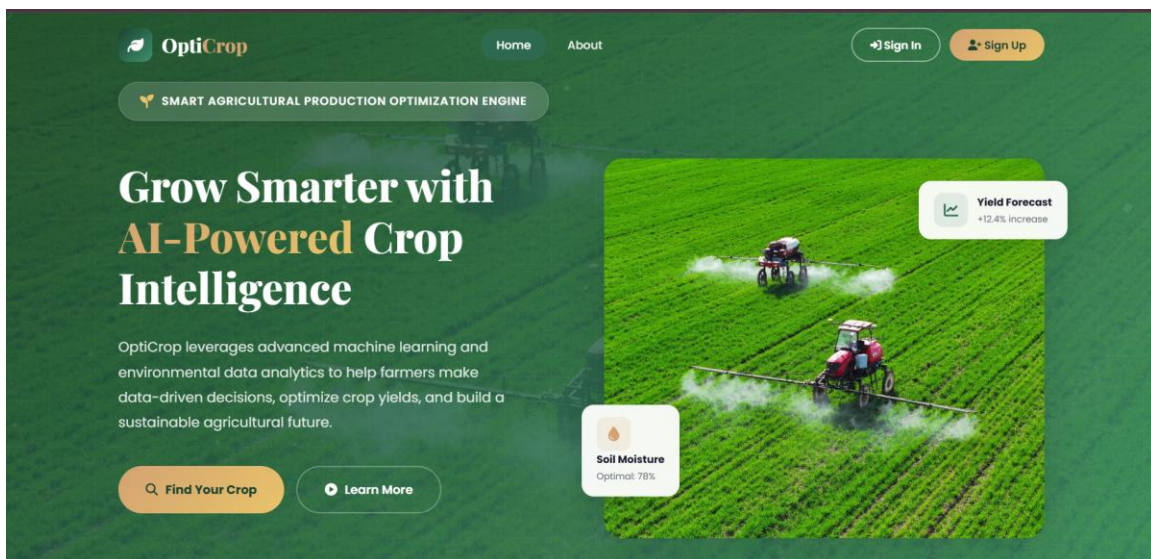
The screenshot shows the 'New Service' configuration page for 'OptiCrop Smart Agricultural Production Optimization Engine'. The form includes fields for Name, Project, Language, Branch, Region, Root Directory, Build Command, and Start Command. The 'Region' dropdown is set to 'Oregon (US West)'.

Field	Value
Name	OptiCrop Smart Agricultural Production Optimization Engine 12
Project	OptiCrop Smart Agricultural Production Optimization Engine / Production
Language	Python 3
Branch	main
Region	Oregon (US West) (11 existing services)
Root Directory	./
Build Command	\$ pip install -r requirements.txt
Start Command	\$ gunicorn app:app

Exploring the Website's Web Pages

Home Page

The Home page serves as the landing page of the OptiCrop application. It introduces the system and provides navigation to the About page, User Registration, User Login, and Admin Login.



About Page

The About page describes the objective of the application and explains how Machine Learning helps recommend suitable crops.

ABOUT OPTICROP

Intelligent Crop Prediction Using Environmental & Soil Parameters

OptiCrop is a Smart Agricultural Production Optimization Engine that uses cutting-edge machine learning techniques to analyze environmental conditions, soil composition, and historical data to recommend the best crops for your farm. Our mission is to empower farmers with actionable insights that improve productivity, reduce waste, and support sustainable agricultural practices.



ML-Powered Analysis
Advanced algorithms for accurate predictions

Climate Integration
Real-time weather & environmental data

Soil Analytics
Comprehensive soil health
assessment

Sustainable Farming
Eco-friendly agricultural practices

→ Explore Features

User Registration Page

Allows new users to create an account by entering their personal details.

Join OptiCrop

Create your account and start optimizing

Full Name

John Doe

Username

johndoe12

Email Address

your@email.com

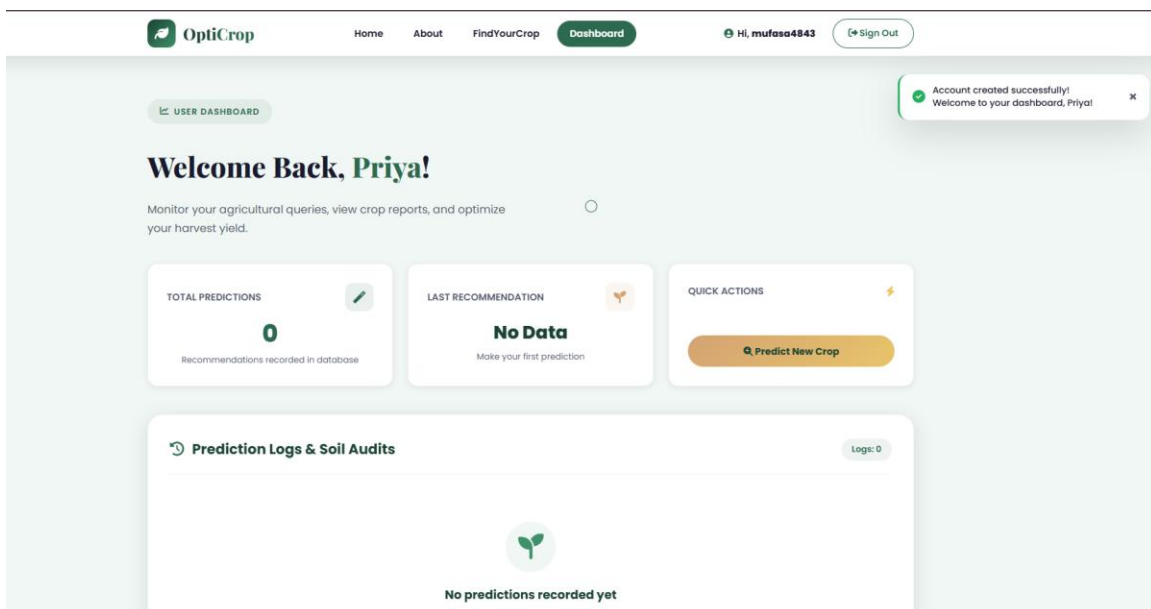
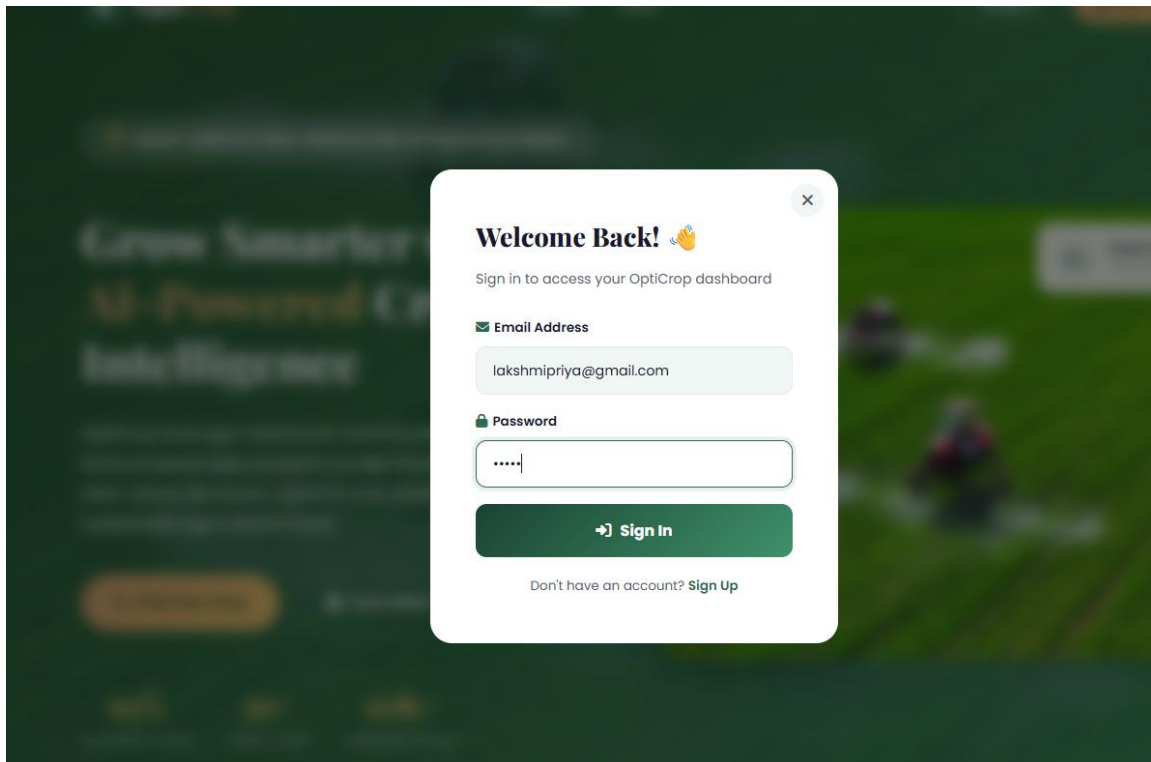
Password

Create Account

Already have an account? [Sign In](#)

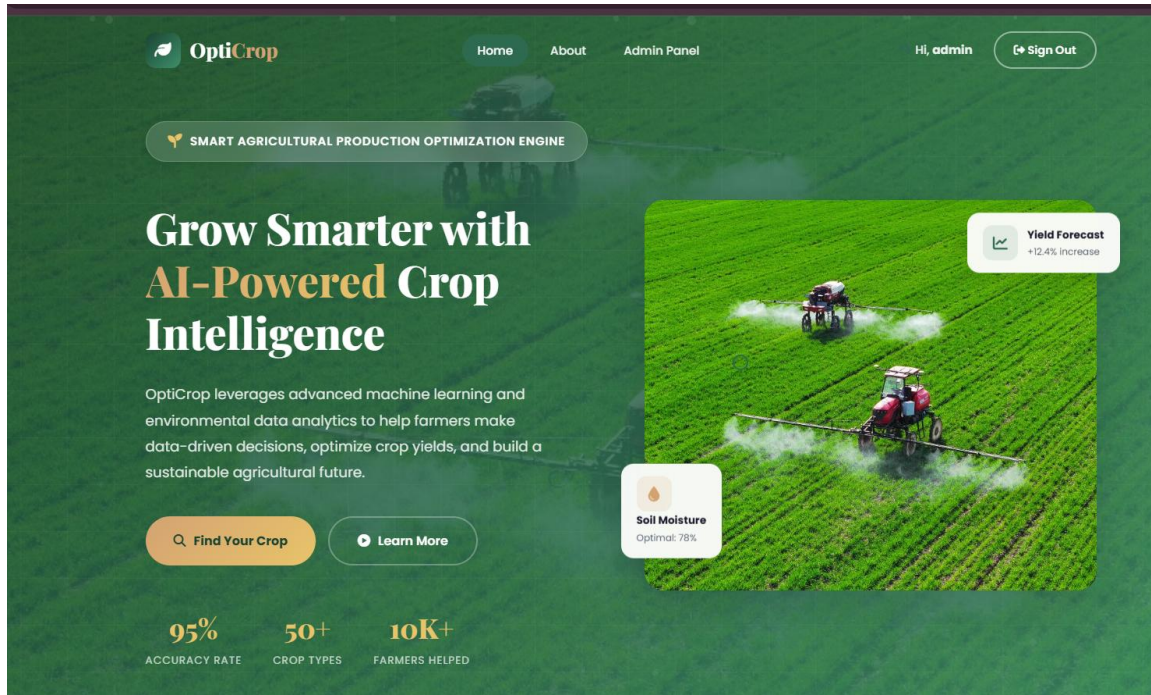
User Login Page

Allows registered users to log into the application securely.



Dashboard

Displays the main dashboard from which users can access the Crop Recommendation module.



Find Your Crop Page

Allows users to enter soil nutrients and environmental parameters required for crop prediction.


[Home](#)
[About](#)
[Admin Panel](#)
Hi, admin
[Sign Out](#)

Q. FIND YOUR CROP

Perfect Crop

Enter your environmental and soil parameters below, and our AI engine will recommend the best crops for optimal yield.

Enter Parameters

Fill in the details about your farming conditions

[Sync Weather Telemetry \(API Key Activated\)](#)

🌡️ Temperature (°C)

💧 Humidity (%)

🌧️ Rainfall (mm)

🌱 Soil pH

🌾 Nitrogen (kg/ha)

🌾 Phosphorus (kg/ha)

🌾 Potassium (kg/ha)

[Predict Best Crop](#)



🌡️ Optimal range: 20-30°C

💧 Ideal: 60-80% humidity

🌧️ Best range: 6.0-7.5

☀️ 6-8 hours daily

Result Page

Displays the crop recommended by the trained Machine Learning model based on the user inputs.


[Home](#)
[About](#)
[Admin Panel](#)
Hi, admin
[Sign Out](#)

📄 PREDICTION REPORT

Agricultural Suitability Report

Review your personalized farm optimization analysis based on our machine learning algorithm.



BEST RECOMMENDED CROP

Chickpea (Gram)

🤖 AI Classifier Confidence **94%**

📊 EXPECTED YIELD

🌱 CROP SEASON

💧 WATER NEEDS

☀️ SUNLIGHT NEEDS

[Print Report](#)
[Dashboard](#)

N

NITROGEN

60 kg/ha

P

PHOSPHORUS

70 kg/ha

K

POTASSIUM

70 kg/ha

⚡

SOIL PH

5.0

🌡️

TEMP

30.0°C

💧

HUMIDITY

60.0%

🌧️

RAINFALL

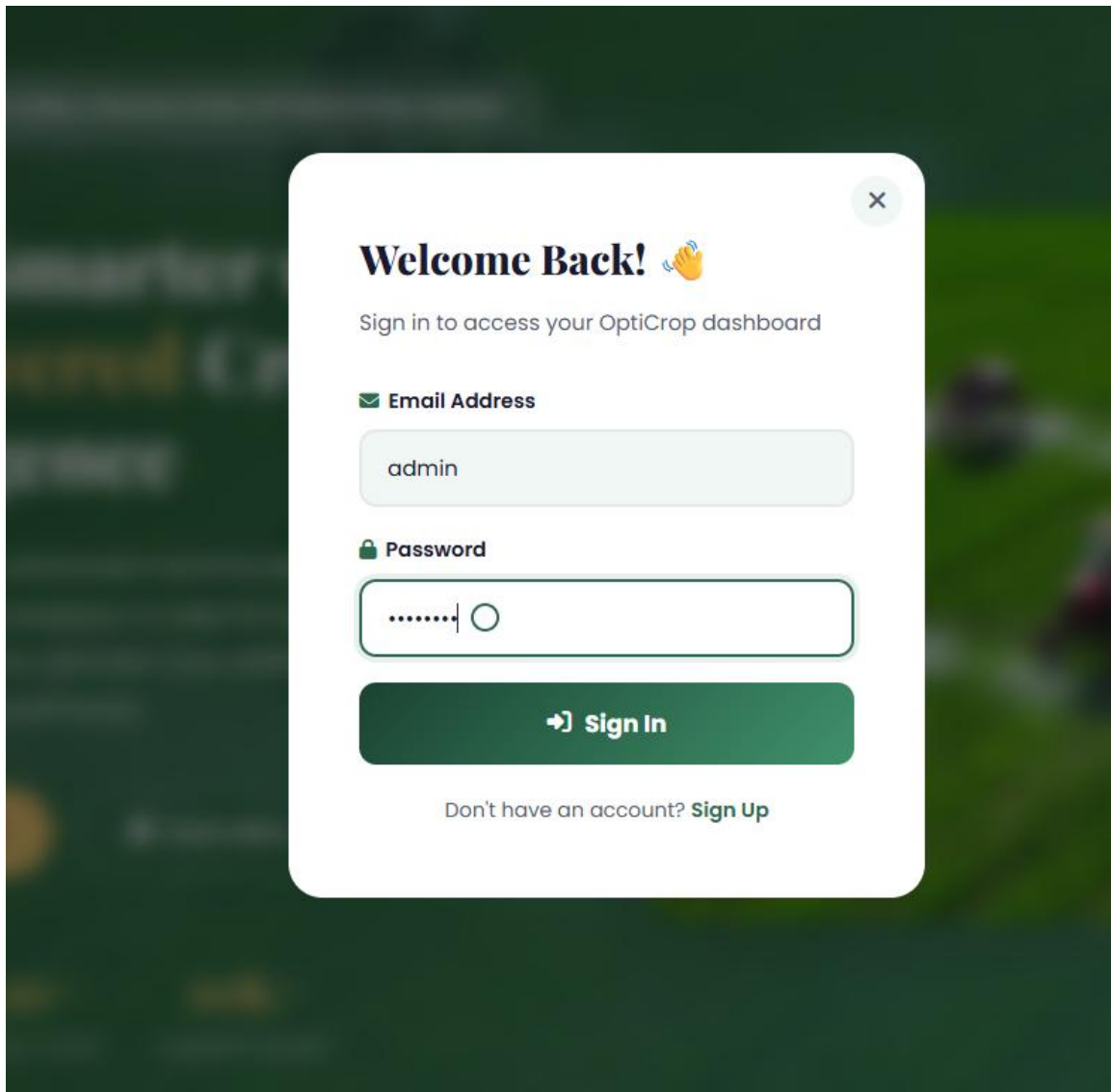
88mm

💡

- Use Rhizobium culture seed treatment to enhance nitrogen fixation.
- Avoid excess irrigation to prevent root rot and vegetative

Admin Login Page

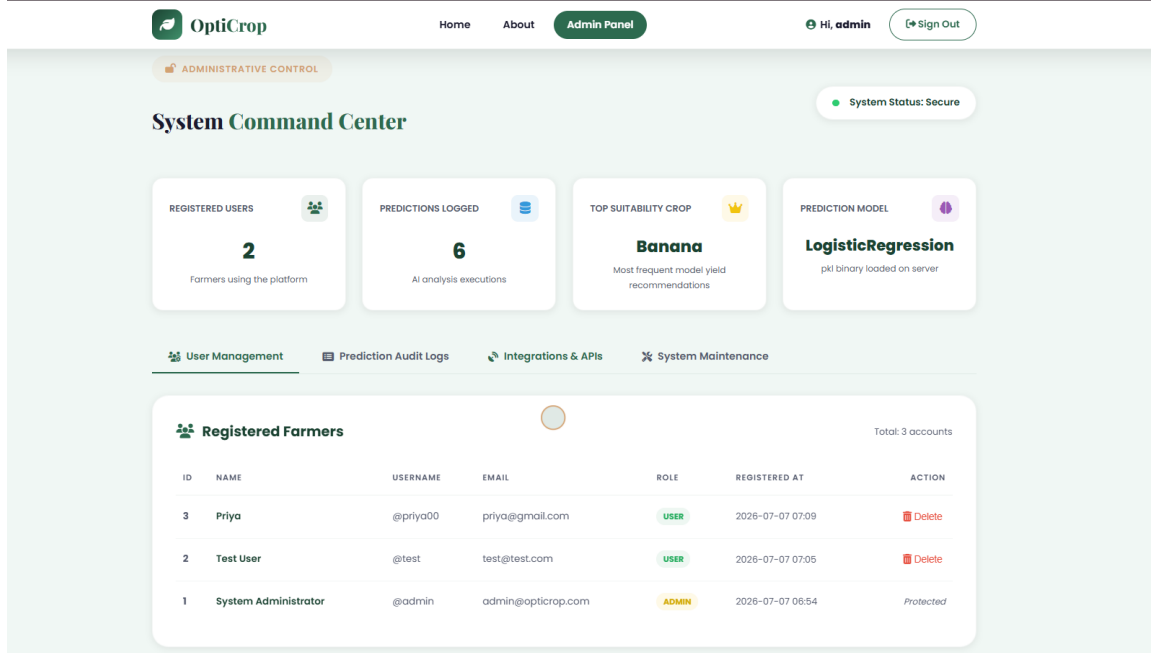
Provides secure administrator authentication.



A screenshot of the Admin Login Page. The page features a dark green background with a blurred image of a field. In the center, there is a white login card with rounded corners. The card has a close button (X) in the top right corner. The text on the card reads: "Welcome Back! 🙌" followed by "Sign in to access your OptiCrop dashboard". Below this, there are two input fields: "Email Address" with a green checkmark icon and "Password" with a green lock icon. The email field contains the text "admin". The password field contains a series of dots and a toggle icon. Below the input fields is a green "Sign In" button with a right arrow icon. At the bottom of the card, there is a link that says "Don't have an account? Sign Up".

Admin Dashboard

Allows administrators to monitor the application and manage registered users.



Conclusion

The **OptiCrop: AI-Based Crop Recommendation System** is designed to support farmers and agricultural professionals in making informed crop selection decisions using Artificial Intelligence and Machine Learning. The application predicts the most suitable crop based on important agricultural parameters such as Nitrogen (N), Phosphorus (P), Potassium (K), Temperature, Humidity, pH value, and Rainfall. By analyzing these parameters through a trained Machine Learning model, the system provides accurate and reliable crop recommendations that can improve agricultural productivity.

The project successfully integrates a user-friendly web interface with a powerful Machine Learning model using the Flask framework. The frontend, developed with HTML, CSS, and JavaScript, provides an interactive environment where users can easily register, log in, and access the crop prediction module. The responsive interface ensures that the application can be used efficiently on different devices, improving accessibility for users.

The Flask backend acts as the core of the application by handling user authentication, managing routing, validating user inputs, and communicating with the trained Machine Learning model. This architecture ensures smooth interaction between the user interface and the prediction engine while maintaining secure and efficient processing of requests.

The trained Random Forest Machine Learning model plays a crucial role in the application by analyzing soil nutrient levels and environmental conditions to recommend the most appropriate crop. Using a pre-trained model stored as

model.pkl allows predictions to be generated quickly without retraining the model during execution, improving the application's performance and scalability.

The inclusion of **User Registration, User Login, Admin Login, Dashboard, Find Your Crop, Result Page, and Admin Dashboard** provides a complete web-based solution rather than a simple prediction tool. These modules enhance usability, security, and application management while providing a structured workflow for both users and administrators.

Deploying the application on **Render** enables users to access the system through a web browser without installing any additional software. This makes the application practical for real-world use and demonstrates how cloud deployment can improve the availability and accessibility of Machine Learning applications.

One of the major advantages of the OptiCrop system is its ability to reduce uncertainty in crop selection by providing data-driven recommendations instead of relying solely on traditional farming practices. This helps improve decision-making, optimize resource utilization, and potentially increase agricultural productivity.

The project also demonstrates the successful integration of Machine Learning with modern web technologies. It provides valuable practical experience in Flask development, frontend design, backend programming, Machine Learning model integration, deployment, and software engineering practices.

Although the current version focuses on crop recommendation, the application can be enhanced further by integrating real-time weather forecasting, soil sensor data, fertilizer recommendations, irrigation planning, disease detection, multilingual support, and mobile application compatibility. These improvements would make the system even more useful for farmers and agricultural organizations.

Overall, the **OptiCrop: AI-Based Crop Recommendation System** successfully achieves its objective of providing an intelligent, reliable, secure, and user-friendly crop recommendation platform. The project demonstrates how Artificial Intelligence and Machine Learning can be effectively applied in agriculture to support sustainable farming practices and improve crop selection decisions. With future enhancements and additional smart farming features, OptiCrop has the potential to become a comprehensive digital agriculture solution that contributes to increased productivity, efficient resource management, and the advancement of precision farming.